

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

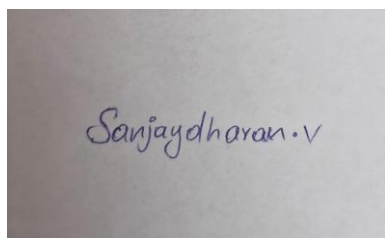
Code of Conduct:

I Sanjay dharan v (192425002) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: x

**Name of the student:
Sanjaydharan v**

A photograph of a piece of paper with the handwritten signature "Sanjaydharan.v" in blue ink.

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Gradient Descent is an optimization algorithm used to minimize the loss function of a machine learning model by updating the model parameters iteratively. The update rule is:

$$\theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t) \quad \theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t)$$

where:

- θ_t = model parameters at iteration t
- η_t = learning rate
- $\nabla J(\theta_t)$ = gradient of the loss function

The learning rate determines the size of each update. Choosing an appropriate learning rate is essential for achieving fast and stable convergence.

High Initial Learning Rate

A high initial learning rate allows the algorithm to make large parameter updates during the early stages of training. This helps the optimizer move quickly toward the region containing the global minimum, especially when the model starts far from the optimal solution. For a convex loss function, this can significantly reduce the number of iterations required in the beginning.

Learning Rate Decay

Instead of keeping the learning rate constant, it is gradually reduced as training progresses. A common decay formula is:

$$\eta_t = \eta_0 \frac{1}{1 + \lambda t} \quad \eta_t = \frac{\eta_0}{1 + \lambda t}$$

where:

- η_0 = initial learning rate
- λ = decay rate
- t = iteration number

As the learning rate decreases, the parameter updates become smaller, allowing the algorithm to fine-tune the solution and converge smoothly to the minimum.

Effect on Convergence

Using a high initial learning rate followed by decay improves convergence by combining two advantages:

1. **Fast initial progress:** Large steps quickly reduce the loss during the early iterations.
2. **Accurate final convergence:** Smaller learning rates near the optimum prevent overshooting and enable precise optimization.

This strategy often converges faster than using a constant learning rate because it balances **speed** and **stability**.

When It Outperforms a Constant Learning Rate

A decaying learning rate performs better than a constant learning rate under the following conditions:

- The loss function is **convex and smooth**.
- The initial parameters are **far from the optimum**.
- The dataset is **large**, requiring efficient optimization.
- High accuracy is required near the final solution.

A constant learning rate may either:

- Be **too small**, causing slow convergence, or
- Be **too large**, leading to oscillations around the minimum.

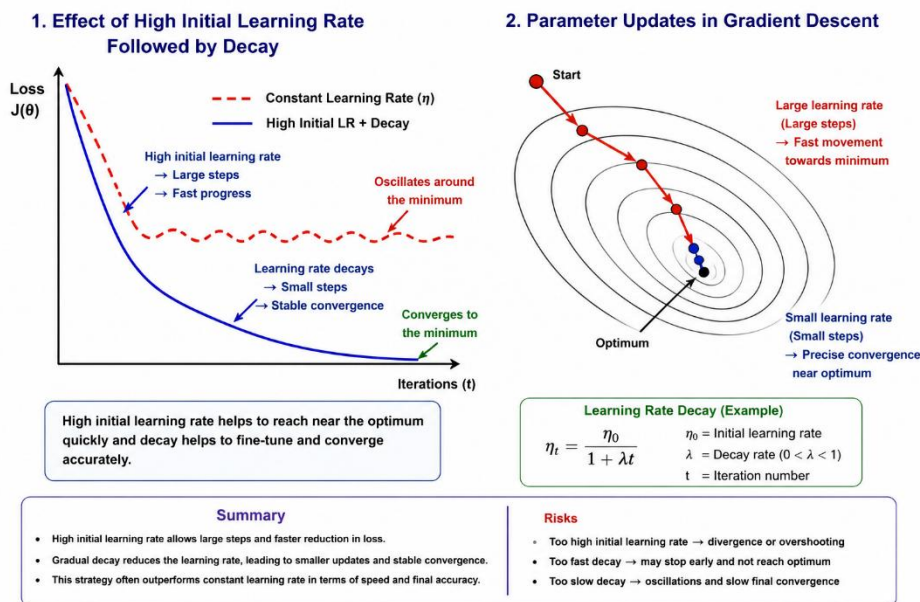
Therefore, a learning rate schedule generally provides better overall performance.

Possible Risks

Despite its advantages, this strategy has some risks:

- If the **initial learning rate is too high**, the algorithm may overshoot the minimum or even diverge.
- If the **learning rate decreases too quickly**, training may stop before reaching the optimal solution.
- If the **decay is too slow**, convergence may remain unstable and oscillate near the minimum.

Hence, selecting appropriate values for the initial learning rate and decay rate is important for successful optimization.



Conclusion

A high initial learning rate followed by gradual decay is an effective optimization strategy because it provides **rapid learning in the early stages** and **stable convergence near the optimum**. For convex loss functions, it often outperforms a constant learning rate by reducing training time while maintaining accuracy. However, careful tuning of the learning rate and decay schedule is necessary to avoid divergence, overshooting, or premature convergence.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

What does this imply about the robustness of MLE?

Answer :

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate unknown parameters of a probability distribution by maximizing the likelihood of the observed data. Assume the observations

$$X = \{x_1, x_2, \dots, x_n\} \quad X = \{x_1, x_2, \dots, x_n\}$$

are independently and identically distributed (i.i.d.) from a Gaussian distribution:

$$X_i \sim N(\mu, \sigma^2) \quad X_i \sim N(\mu, \sigma^2)$$

where:

- μ = unknown mean
- σ^2 = known variance
- n = number of observations

Step 1: Likelihood Function

The probability density function of a Gaussian distribution is

$$f(x_i|\mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Since all observations are independent, the likelihood function is

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Step 2: Log-Likelihood Function

Taking the natural logarithm,

$$\ell(\mu) = \ln L(\mu) = \ln \left(\prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right) \right)$$

$$\ell(\mu) = -n \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3: Differentiate and Set Equal to Zero

Differentiate with respect to μ :

$$\frac{d\ell(\mu)}{d\mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu)$$

Set

$$\frac{d\ell(\mu)}{d\mu} = 0 \Rightarrow \sum_{i=1}^n (x_i - \mu) = 0$$

Then,

$$\sum_{i=1}^n (x_i - \mu) = 0 \Rightarrow \sum_{i=1}^n x_i - n\mu = 0 \Rightarrow \mu = \frac{1}{n} \sum_{i=1}^n x_i$$

Hence, the Maximum Likelihood Estimator is

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

Thus, the MLE of the mean is simply the sample mean.

Effect of Outliers

An outlier is an observation that is much larger or smaller than the remaining data.

Suppose the dataset is

$$\{10, 12, 11, 13, 100\}$$

The estimated mean becomes

$$\hat{\mu} = \frac{10+12+11+13+100}{5} = 29.2$$

Although four observations are close to **12**, one extreme value (**100**) shifts the estimated mean to **29.2**.

Reason:

Since every observation contributes equally to

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

even one extreme value can significantly affect the estimate.

Hence, **MLE for the Gaussian mean is sensitive to outliers.**

Effect of Increasing Sample Size

As the sample size n increases,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

approaches the true population mean.

According to the **Law of Large Numbers**,

$$\hat{\mu} \xrightarrow[n \rightarrow \infty]{} \mu$$

Also, the variance of the estimator decreases:

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}$$

Therefore,

- Larger datasets produce **more accurate estimates**.
 - Random fluctuations reduce.
 - The estimator becomes more stable and reliable.
-

Robustness of MLE

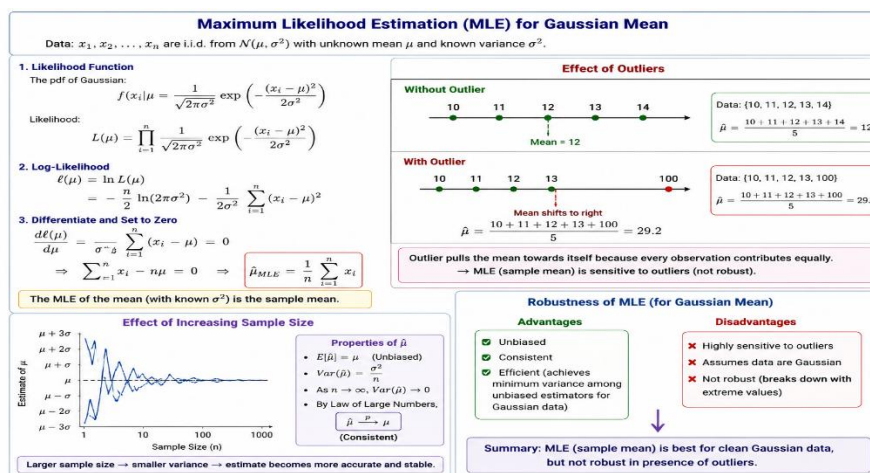
Robustness refers to the ability of an estimator to remain accurate even when the data contain outliers or small deviations from assumptions.

For a Gaussian distribution,

- **Advantage:** MLE is **unbiased**, **consistent**, and **efficient** for normally distributed data.
- **Disadvantage:** It is **not robust** because the sample mean is highly influenced by extreme observations.

Therefore,

- With **clean Gaussian data**, MLE gives the best estimate.
- With **outliers**, the estimate can become inaccurate.
- In such cases, robust estimators such as the **median** or **trimmed mean** perform better.



Conclusion

The Maximum Likelihood Estimator for the mean of a Gaussian distribution with known variance is

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

It is obtained by maximizing the likelihood function and is equal to the sample mean. As the sample size increases, the estimator converges to the true mean and its variance decreases, making it more accurate. However, because every observation contributes equally, the estimator is highly sensitive to outliers. Hence, MLE is **efficient and consistent for Gaussian data but not robust in the presence of extreme values**.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

In this problem, the dataset contains 10,000 features but only 500 training samples. This is known as a high-dimensional, low-sample-size (HDLSS) problem. Since the number of features is much larger than the number of samples, the model can easily memorize the training data instead of learning general patterns, leading to overfitting.

Challenges

The main challenges are:

- **Curse of Dimensionality:** High-dimensional data makes learning difficult because data points become sparse.
 - **Overfitting:** The model learns noise instead of meaningful patterns.
 - **High Computational Cost:** Training becomes slower with many features.
 - **Irrelevant and Redundant Features:** Many features may not contribute to prediction.
 - **Poor Generalization:** The model performs well on training data but poorly on unseen test data.
-

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Standardization:

$$z = \frac{x - \mu}{\sigma}$$

where

- x = feature value
- μ = mean
- σ = standard deviation

This ensures that all features have similar scales.

Step 2: Feature Selection

Select only the most informative features using methods such as

- Mutual Information
- Chi-Square Test
- Recursive Feature Elimination (RFE)
- L1 (Lasso) Feature Selection

Example:

Reduce

10000 → 500

important features.

This removes noisy and irrelevant variables.

Step 3: Dimensionality Reduction

Apply Principal Component Analysis (PCA) to further reduce dimensionality.

The covariance matrix is

$$C = \frac{1}{n} X^T X$$

PCA projects data onto principal components that retain maximum variance.

Example:

500 → 50

principal components.

Benefits:

- Removes redundancy
 - Reduces computation
 - Preserves important information
-

Step 4: Train a Regularized Classifier

Use a classifier such as

- Logistic Regression with L2 Regularization
- Support Vector Machine (Linear SVM)

- Random Forest (if appropriate)

Regularized Logistic Regression minimizes

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h(\theta(x_i))) + (1-y_i) \log(1-h(\theta(x_i)))] + \frac{\lambda}{2} \|\theta\|^2$$

where

- λ = regularization parameter

Regularization penalizes large weights and reduces model complexity.

Step 5: Cross Validation

Use k-fold Cross Validation (e.g., k=5).

- Split data into 5 folds.
- Train on 4 folds.
- Validate on 1 fold.
- Repeat five times.

This provides a reliable estimate of model performance and helps choose optimal hyperparameters.

How the Pipeline Mitigates Overfitting

Each stage helps reduce overfitting:

Step	Purpose
Feature Selection	Removes irrelevant and noisy features
PCA	Reduces dimensionality while preserving important information
Regularization	Prevents excessively large model weights
Cross Validation	Detects overfitting and improves model selection
Standardization	Ensures stable optimization and faster convergence

As a result,

- Model complexity decreases.

- Noise is reduced.
 - Generalization improves.
 - Prediction accuracy on unseen data increases.
-

Algorithm Pipeline

Raw Dataset

(10,000 Features, 500 Samples)

|



Data Preprocessing

(Standardization, Missing Values)

|



Feature Selection

(10,000 → 500 Features)

|



PCA

(500 → 50 Components)

|



Regularized Classifier

(Logistic Regression / Linear SVM)

|



Cross Validation

|



Final Trained Model

Conclusion

For datasets with many features but few samples, directly training a classifier leads to severe overfitting. An effective strategy is to preprocess the data, perform feature selection, apply PCA for dimensionality reduction, train a regularized classifier, and validate using k-fold cross validation. This pipeline reduces model complexity, removes irrelevant information, improves computational efficiency, and enhances generalization, resulting in a more accurate and robust classification model.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Question: Consider a Multilayer Perceptron (MLP) with one hidden layer using sigmoid activation functions. Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

A Multilayer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron in the hidden layer applies a nonlinear activation function, allowing the network to learn complex relationships between inputs and outputs.

For a hidden neuron with sigmoid activation,

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where

$$z = \sum_{i=1}^n w_{ix} x_i + b_z$$

Here,

- x_i = input features
- w_{ix} = weights
- b_z = bias

- $\sigma(z)$ sigmoid activation

The output of an MLP with one hidden layer is

$$\hat{y} = \sum_{j=1}^m v_j \sigma\left(\sum_{i=1}^n w_{ij} x_i + b_j\right) + c$$

where

- m = number of hidden neurons
- v_j = output weights
- c = output bias

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns a different nonlinear feature of the input data. Increasing the number of hidden neurons increases the number of nonlinear transformations that the network can combine.

As the number of hidden neurons (m) increases,

- More decision boundaries can be formed.
- More complex input-output relationships can be represented.
- The network can approximate highly nonlinear functions.

According to the Universal Approximation Theorem, an MLP with a sufficient number of hidden neurons can approximate any continuous function with arbitrary accuracy.

Therefore,

More hidden neurons $\Rightarrow$ Higher representational power.

Advantages of Increasing Hidden Neurons

- Learns complex patterns.
- Improves feature representation.
- Reduces training error.
- Models nonlinear decision boundaries.
- Can solve difficult classification problems.

Why More Hidden Neurons Do Not Always Improve Performance

Although additional neurons increase model capacity, they also increase the number of trainable parameters.

A larger network may begin to memorize the training data instead of learning general patterns.

This phenomenon is called overfitting.

In overfitting,

- Training accuracy becomes very high.
- Test accuracy decreases.
- The model learns noise instead of useful information.

Hence, increasing hidden neurons beyond an optimal number can reduce real-world performance.

Generalization

Generalization is the ability of a model to perform well on unseen data.

A good model should have

- Low training error
- Low testing error

If the network is too simple,

→ Underfitting

If the network is too complex,

→ Overfitting

The best performance is achieved by selecting an appropriate number of hidden neurons that balances model complexity and prediction accuracy.

How to Prevent Overfitting

Several techniques improve generalization:

- Regularization (L1/L2): Penalizes large weights.
- Dropout: Randomly deactivates neurons during training.
- Early Stopping: Stops training before overfitting begins.
- Cross Validation: Selects the optimal network size.

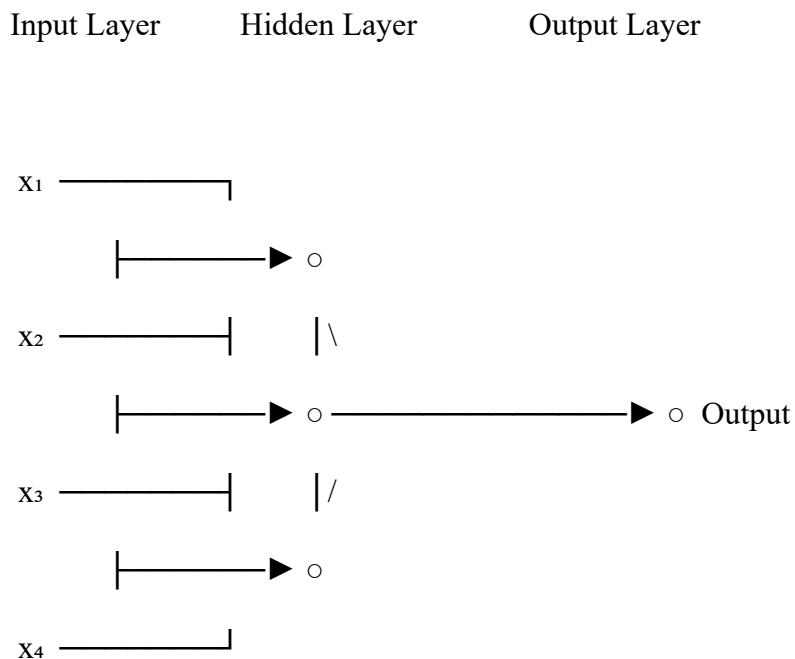
- More Training Data: Improves learning and reduces variance.

Comparison

Few Hidden Neurons	Too Many Hidden Neurons
Low model complexity	High model complexity
May underfit	May overfit
High bias	High variance
Poor representation	Excellent representation but poor generalization
Lower training accuracy	Very high training accuracy

Diagram

Multilayer Perceptron (MLP)



More hidden neurons



Higher representational power



Learns more complex patterns



If too many neurons



Overfitting



Poor Generalization

Conclusion

Adding more hidden neurons in an MLP increases its representational power because each neuron learns a different nonlinear feature, enabling the network to approximate more complex functions. However, increasing the number of neurons also increases model complexity and the risk of overfitting, where the network memorizes the training data instead of generalizing to unseen data. Therefore, the best-performing MLP balances model capacity and generalization using an appropriate number of hidden neurons along with regularization techniques such as dropout, early stopping, and cross-validation.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

A Deep Neural Network (DNN) is trained by minimizing a loss function using Backpropagation and Stochastic Gradient Descent (SGD). When the dataset contains a very large number of features, the model suffers from the curse of dimensionality, where the feature space becomes sparse and learning becomes more difficult.

Backpropagation and SGD

In backpropagation, gradients are computed using the chain rule and weights are updated using SGD.

The SGD update rule is

$$w_{t+1} = w_t - \eta \nabla J(w_t) \quad w_{t+1} = w_t - \eta \nabla J(w_t)$$

where

- w_t = current weight
- η = learning rate
- $\nabla J(w_t)$ = gradient of the loss function

The objective is to minimize the loss $J(w)$.

Impact on Gradient Updates in Backpropagation

In high-dimensional datasets,

- The number of weights becomes very large.
- Gradients are computed for thousands or millions of parameters.
- Many features contribute very little useful information.

As a result,

- Gradient estimates become noisy.
- Some gradients become very small (vanishing gradients) or very large (exploding gradients).
- Weight updates become unstable.

Hence, backpropagation becomes less efficient.

Impact on Convergence Speed of SGD

SGD updates parameters using one training sample (or a small batch) at a time.

In high dimensions,

- More parameters must be optimized.
- The optimization landscape becomes more complex.
- SGD may oscillate before reaching the minimum.
- More iterations are required.

Therefore,

- Training time increases.

- Convergence becomes slower.
- Computational cost increases.

Impact on Model Generalization

When the number of features is much larger than the number of training samples, the network can memorize the training data instead of learning useful patterns.

This leads to

- High training accuracy
- Poor testing accuracy

This phenomenon is known as overfitting.

Thus, the model fails to generalize well to unseen data.

Techniques to Overcome the Curse of Dimensionality

1. Dimensionality Reduction (PCA)

Apply Principal Component Analysis (PCA) before training.

Covariance matrix:

$$C = \frac{1}{n} X^T X$$

PCA transforms the original features into a smaller number of principal components.

Benefits:

- Removes redundant features.
- Reduces computation.
- Improves convergence.
- Reduces noise.

Example:

10000 features → 300 principal components

2. Regularization

Use L2 Regularization (Weight Decay).

The new loss function becomes

$$J_{\text{new}} = J(w) + \lambda \|w\|^2 \quad J_{\text{new}} = J(w) + \lambda \|w\|^2$$

where

- λ = regularization parameter

Regularization penalizes large weights.

Benefits:

- Reduces model complexity.
 - Prevents overfitting.
 - Improves generalization.
-

3. Dropout (Optional Additional Technique)

During training,

a fraction of neurons is randomly deactivated.

Benefits:

- Prevents co-adaptation of neurons.
 - Forces the network to learn robust features.
 - Improves test accuracy.
-

4. Mini-Batch SGD (Optional Additional Technique)

Instead of using one sample,

update weights using a small batch.

Benefits:

- Less noisy gradients.
 - Faster convergence.
 - More stable optimization.
-

Summary of Solutions

Technique	Benefit
PCA	Reduces dimensionality and computation
L2 Regularization	Prevents overfitting
Dropout	Improves generalization
Mini-Batch SGD	Faster and smoother convergence

Algorithm Flow Diagram

High-Dimensional Dataset

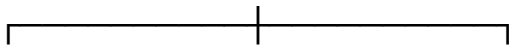
(Thousands of Features)

|



Curse of Dimensionality

|



| | |



Noisy Slow SGD Overfitting

Gradients Convergence Poor Generalization

|



Apply PCA + Regularization

|



Reduced Features + Smaller Weights

|



Faster Convergence

Better Generalization

Conclusion

The curse of dimensionality negatively affects gradient updates, SGD convergence, and model generalization in deep neural networks. High-dimensional data increases the number of trainable parameters, making optimization slower and increasing the risk of overfitting. Applying Principal Component Analysis (PCA) reduces the number of features, while L2 regularization limits model complexity by penalizing large weights. Techniques such as dropout and mini-batch SGD further improve convergence and generalization. Together, these methods produce a more efficient, accurate, and robust deep learning model suitable for high-dimensional datasets.